O'REILLY

Chapter 11. *Performance and Scalability*

One of the primary reasons to use threads is to improve performance.[1] Using threads can improve resource utilization by letting applications more easily exploit available processing capacity, and can improve responsiveness by letting applications begin processing new tasks immediately while existing tasks are still running.

[1] Some might argue this is the *only* reason we put up with the complexity threads introduce.

This chapter explores techniques for analyzing, monitoring, and improving the performance of concurrent programs. Unfortunately, many of the techniques for improving performance also increase complexity, thus increasing the likelihood of safety and liveness failures. Worse, some techniques intended to improve performance are actually counterproductive or trade one sort of performance problem for another. While better performance is often desirable—and improving performance can be very satisfying—safety always comes first. First make your program right, then make it fast—and then only if your performance requirements and measurements tell you it needs to be faster. In designing a concurrent application, squeezing out the last bit of performance is often the least of your concerns.

11.1. Thinking about Performance

Improving performance means doing more work with fewer resources. The meaning of "resources" can vary; for a given activity, some specific resource is usually in shortest supply, whether it is CPU cycles, memory, network bandwidth, I/O bandwidth, database requests, disk space, or any number of other resources. When the performance of an activity is limited by availability of a particular resource, we say it is *bound* by that resource: CPU-bound, database-bound, etc.

While the goal may be to improve performance overall, using multiple threads always introduces some performance costs compared to the single-threaded approach. These include the overhead associated with coordinating between threads (locking, signaling, and memory synchronization), increased context switching, thread creation and teardown, and scheduling overhead. When threading is employed effectively, these costs are more than made up for by greater throughput, responsiveness, or capacity. On the other hand, a poorly designed concurrent application can perform even worse than a comparable sequential one.[2]

[2] A colleague provided this amusing anecodote: he had been involved in the testing of an expensive and complex application that managed its work via a tunable thread pool. After the system was complete, testing showed that the optimal number of threads for the pool was . . . 1. This should have been obvious from the outset; the target system was a single-CPU system and the application was almost entirely CPU-bound.

In using concurrency to achieve better performance, we are trying to do two things: utilize the processing resources we have more effectively, and enable our program to exploit additional processing resources if they become available. From a performance monitoring perspective, this means we are looking to keep the CPUs as busy as possible. (Of course, this doesn't mean burning cycles with useless computation; we want to keep the CPUs busy with *useful* work.) If the program is compute-bound, then we may be able to increase its capacity by adding more processors; if it can't even keep the processors we have busy, adding more won't help. Threading offers a means to keep the CPU(s) “hotter” by decomposing the application so there is always work to be done by an available processor.

11.1.1. Performance Versus Scalability

Application performance can be measured in a number of ways, such as service time, latency, throughput, efficiency, scalability, or capacity. Some of these (service time, latency) are measures of “how fast” a given unit of work can be processed or acknowledged; others (capacity, throughput) are measures of “how much” work can be performed with a given quantity of computing resources.

Scalability describes the ability to improve throughput or capacity when additional computing resources (such as additional CPUs, memory, storage, or I/O bandwidth) are added.

Designing and tuning concurrent applications for scalability can be very different from traditional performance optimization. When tuning for performance, the goal is usually to do the *same* work with *less* effort, such as by reusing previously computed results through caching or replacing an $O(n^2)$ algorithm with an $O(n \log n)$ one. When tuning for scalability, you are instead trying to find ways to parallelize the problem so you can take advantage of additional processing resources to do *more* work with *more* resources.

These two aspects of performance—*how fast* and *how much*—are completely separate, and sometimes even at odds with each other. In order to achieve higher scalability or better hardware utilization, we often end up *increasing* the amount of work done to process each *individual* task, such as when we divide tasks into multiple "pipelined" subtasks. Ironically, many of the tricks that improve performance in single-threaded programs are bad for scalability (see **Section 11.4.4** for an example).

The familiar three-tier application model—in which presentation, business logic, and persistence are separated and may be handled by different systems—illustrates how improvements in scalability often come at the expense of performance. A monolithic application where presentation, business logic, and persistence are intertwined would almost certainly provide better performance for the *first* unit of work than would a well-factored multitier implementation distributed over multiple systems. How could it not? The monolithic application would not have the network latency inherent in handing off tasks between tiers, nor would it have to pay the costs inherent in separating a computational process into distinct abstracted layers (such as queuing overhead, coordination overhead, and data copying).

However, when the monolithic system reaches its processing capacity, we could have a serious problem: it may be prohibitively difficult to signifi-

cantly increase capacity. So we often accept the performance costs of longer service time or greater computing resources used per unit of work so that our application can scale to handle greater load by adding more resources.

Of the various aspects of performance, the "how much" aspects—scalability, throughput, and capacity—are usually of greater concern for server applications than the "how fast" aspects. (For interactive applications, latency tends to be more important, so that users need not wait for indications of progress and wonder what is going on.) This chapter focuses primarily on scalability rather than raw single-threaded performance.

11.1.2. Evaluating Performance Tradeoffs

Nearly all engineering decisions involve some form of tradeoff. Using thicker steel in a bridge span may increase its capacity and safety, but also its construction cost. While software engineering decisions don't usually involve tradeoffs between money and risk to human life, we often have less information with which to make the right tradeoffs. For example, the "quicksort" algorithm is highly efficient for large data sets, but the less sophisticated "bubble sort" is actually more efficient for small data sets. If you are asked to implement an efficient sort routine, you need to know something about the sizes of data sets it will have to process, along with metrics that tell you whether you are trying to optimize average-case time, worst-case time, or predictability. Unfortunately, that information is often not part of the requirements given to the author of a library sort routine. This is one of the reasons why most optimizations are premature: *they are often undertaken before a clear set of requirements is available*.

Avoid premature optimization. First make it right, then make it fast—*if* it is not already fast enough.

When making engineering decisions, sometimes you are trading one form of cost for another (service time versus memory consumption); sometimes you are trading cost for safety. Safety doesn't necessarily mean

risk to human lives, as it did in the bridge example. Many performance optimizations come at the cost of readability or maintainability—the more "clever" or nonobvious code is, the harder it is to understand and maintain. Sometimes optimizations entail compromising good object-oriented design principles, such as breaking encapsulation; sometimes they involve greater risk of error, because faster algorithms are usually more complicated. (If you can't spot the costs or risks, you probably haven't thought it through carefully enough to proceed.)

Most performance decisions involve multiple variables and are highly situational. Before deciding that one approach is "faster" than another, ask yourself some questions:

- What do you mean by "faster"?
- Under what conditions will this approach *actually* be faster? Under light or heavy load? With large or small data sets? Can you support your answer with measurements?
- How often are these conditions likely to arise in your situation? Can you support your answer with measurements?
- Is this code likely to be used in other situations where the conditions may be different?
- What hidden costs, such as increased development or maintenance risk, are you trading for this improved performance? Is this a good tradeoff?

These considerations apply to any performance-related engineering decision, but this is a book about concurrency. Why are we recommending such a conservative approach to optimization? *The quest for performance is probably the single greatest source of concurrency bugs.* The belief that synchronization was "too slow" led to many clever-looking but dangerous idioms for reducing synchronization (such as double-checked locking, discussed in **Section 16.2.4**), and is often cited as an excuse for not following the rules regarding synchronization. Because concurrency bugs are among the most difficult to track down and eliminate, however, anything that risks introducing them must be undertaken very carefully.

Worse, when you trade safety for performance, you may get neither. Especially when it comes to concurrency, the intuition of many develop-

ers about where a performance problem lies or which approach will be faster or more scalable is often incorrect. It is therefore imperative that any performance tuning exercise be accompanied by concrete performance requirements (so you know both when to tune and when to *stop* tuning) and with a measurement program in place using a realistic configuration and load profile. Measure again after tuning to verify that you've achieved the desired improvements. The safety and maintenance risks associated with many optimizations are bad enough—you don't want to pay these costs if you don't need to—and you definitely don't want to pay them if you don't even get the desired benefit.

Measure, don't guess.

There are sophisticated profiling tools on the market for measuring performance and tracking down performance bottlenecks, but you don't have to spend a lot of money to figure out what your program is doing. For example, the free `perfbar` application can give you a good picture of how busy the CPUs are, and since your goal is usually to keep the CPUs busy, this is a very good way to evaluate whether you need performance tuning or how effective your tuning has been.

11.2. Amdahl's Law

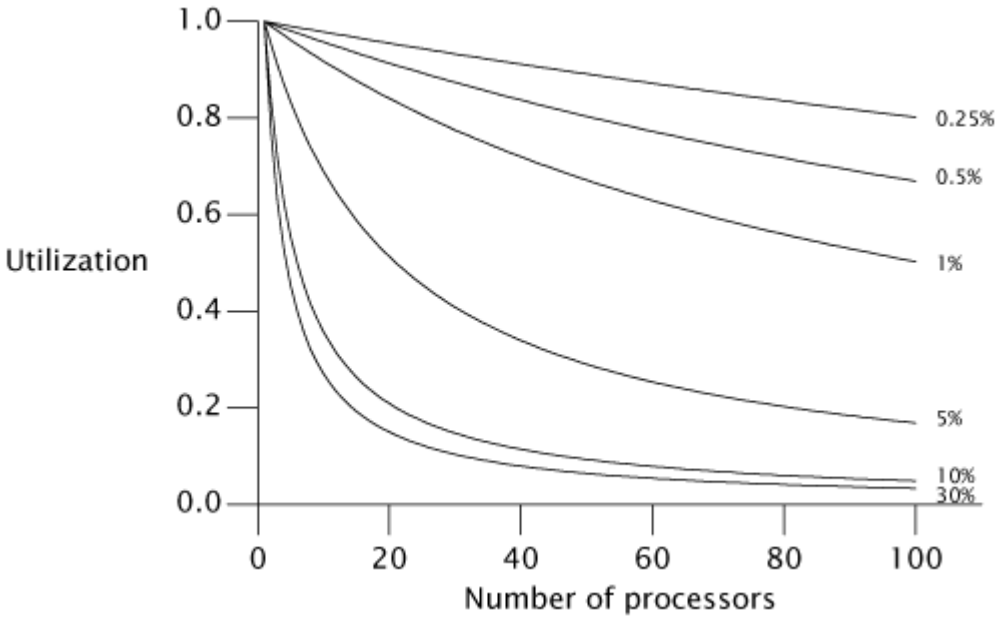
Some problems can be solved faster with more resources—the more workers available for harvesting crops, the faster the harvest can be completed. Other tasks are fundamentally serial—no number of additional workers will make the crops grow any faster. If one of our primary reasons for using threads is to harness the power of multiple processors, we must also ensure that the problem is amenable to parallel decomposition and that our program effectively exploits this potential for parallelization.

Most concurrent programs have a lot in common with farming, consisting of a mix of parallelizable and serial portions. *Amdahl's law* describes how much a program can theoretically be sped up by additional computing resources, based on the proportion of parallelizable and serial components.

If *F* is the fraction of the calculation that must be executed serially, then Amdahl's law says that on a machine with *N* processors, we can achieve a speedup of at most:

$$Speedup \leq \frac{1}{F + \frac{(1-F)}{N}}$$

As *N* approaches infinity, the maximum speedup converges to 1/*F*, meaning that a program in which fifty percent of the processing must be executed serially can be sped up only by a factor of two, regardless of how many processors are available, and a program in which ten percent must be executed serially can be sped up by at most a factor of ten. Amdahl's law also quantifies the efficiency cost of serialization. With ten processors, a program with 10%serialization can achieve at most a speedup of 5.3 (at 53% utilization), and with 100 processors it can achieve at most a speedup of 9.2 (at 9% utilization). It takes a lot of inefficiently utilized CPUs to never get to that factor of ten.



Chapter 6 explored identifying logical boundaries for decomposing applications into tasks. But in order to predict what kind of speedup is possible from running your application on a multiprocessor system, you also need to identify the sources of serialization in your tasks.

Imagine an application where *N* threads execute `doWork` in **Listing 11.1**, fetching tasks from a shared work queue and processing them; assume

that tasks do not depend on the results or side effects of other tasks. Ignoring for a moment how the tasks get onto the queue, how well will this application scale as we add processors? At first glance, it may appear that the application is completely parallelizable: tasks do not wait for each other, and the more processors available, the more tasks can be processed concurrently. However, there is a serial component as well—fetching the task from the work queue. The work queue is shared by all the worker threads, and it will require some amount of synchronization to maintain its integrity in the face of concurrent access. If locking is used to guard the state of the queue, then while one thread is dequeing a task, other threads that need to dequeue their next task must wait—and this is where task processing is serialized.

The processing time of a single task includes not only the time to execute the task `Runnable`, but also the time to dequeue the task from the shared work queue. If the work queue is a `LinkedBlockingQueue`, the dequeue operation may block less than with a synchronized `LinkedList` because `LinkedBlockingQueue` uses a more scalable algorithm, but accessing any shared data structure fundamentally introduces an element of serialization into a program.

This example also ignores another common source of serialization: result handling. All useful computations produce some sort of result or side effect—if not, they can be eliminated as dead code. Since `Runnable` provides for no explicit result handling, these tasks must have some sort of side effect, say writing their results to a log file or putting them in a data structure. Log files and result containers are usually shared by multiple worker threads and therefore are also a source of serialization. If instead each thread maintains its own data structure for results that are merged after all the tasks are performed, then the final merge is a source of serialization.

Listing 11.1. Serialized Access to a Task Queue.

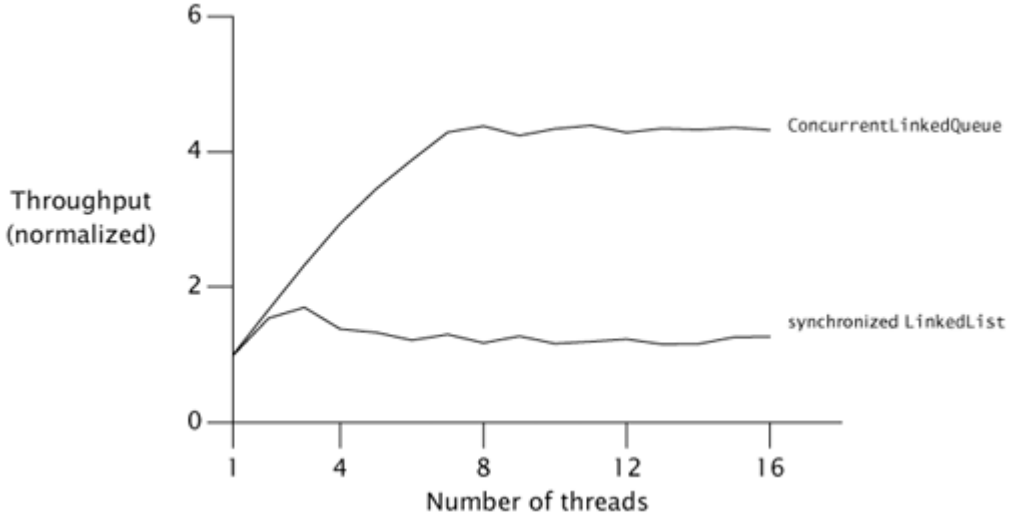
```
public class WorkerThread extends Thread {
    private final BlockingQueue<Runnable> queue;

    public WorkerThread(BlockingQueue<Runnable> queue) {
        this.queue = queue;
    }

    public void run() {
        while (true) {
            try {
                Runnable task = queue.take();
                task.run();
            } catch (InterruptedException e) {
                break; /* Allow thread to exit */
            }
        }
    }
}
```

All concurrent applications have some sources of serialization; if you think yours does not, look again.

11.2.1. Example: Serialization Hidden in Frameworks

To see how serialization can be hidden in the structure of an application, we can compare throughput as threads are added and infer differences in serialization based on observed differences in scalability. **Figure 11.2** shows a simple application in which multiple threads repeatedly remove an element from a shared `Queue` and process it, similar to **Listing 11.1**. The processing step involves only thread-local computation. If a thread finds the queue is empty, it puts a batch of new elements on the queue so that other threads have something to process on their next iteration. Accessing the shared queue clearly entails some degree of serialization, but the processing step is entirely parallelizable since it involves no shared data.

Figure 11.2. Comparing Queue Implementations.



The curves in **Figure 11.2** compare throughput for two thread-safe `Queue` implementations: a `LinkedList` wrapped with `synchronized-List`, and a `ConcurrentLinkedQueue`. The tests were run on an 8-way Sparc V880 system running Solaris. While each run represents the same amount of "work", we can see that merely changing queue implementations can have a big impact on scalability.

The throughput of `ConcurrentLinkedQueue` continues to improve until it hits the number of processors and then remains mostly constant. On the other hand, the throughput of the synchronized `LinkedList` shows some improvement up to three threads, but then falls off as synchronization overhead increases. By the time it gets to four or five threads, contention is so heavy that every access to the queue lock is contended and throughput is dominated by context switching.

The difference in throughput comes from differing degrees of serialization between the two queue implementations. The synchronized `LinkedList` guards the entire queue state with a single lock that is held for the duration of the `offer` or `remove` call; `ConcurrentLinkedQueue` uses a sophisticated nonblocking queue algorithm (see **Section 15.4.2**) that uses atomic references to update individual link pointers. In one, the entire insertion or removal is serialized; in the other, only updates to individual pointers are serialized.

11.2.2. Applying Amdahl's Law Qualitatively

Amdahl's law quantifies the possible speedup when more computing resources are available, if we can accurately estimate the fraction of execution that is serialized. Although measuring serialization directly can be difficult, Amdahl's law can still be useful without such measurement.

Since our mental models are influenced by our environment, many of us are used to thinking that a multiprocessor system has two or four processors, or maybe (if we've got a big budget) as many as a few dozen, because this is the technology that has been widely available in recent years. But as multicore CPUs become mainstream, systems will have hundreds or even thousands of processors. [3] Algorithms that seem scalable on a four-way system may have hidden scalability bottlenecks that have just not yet been encountered.

[3] Market update: at this writing, Sun is shipping low-end server systems based on the 8-core Niagara processor, and Azul is shipping high-end server systems (96, 192, and 384-way) based on the 24-core Vega processor.

When evaluating an algorithm, thinking “in the limit” about what would happen with hundreds or thousands of processors can offer some insight into where scaling limits might appear. For example, **Sections 11.4.2** and **11.4.3** discuss two techniques for reducing lock granularity: lock splitting (splitting one lock into two) and lock striping (splitting one lock into many). Looking at them through the lens of Amdahl's law, we see that splitting a lock in two does not get us very far towards exploiting many processors, but lock striping seems much more promising because the size of the stripe set can be increased as processor count increases. (Of course, performance optimizations should always be considered in light of actual performance requirements; in some cases, splitting a lock in two may be enough to meet the requirements.)

11.3. Costs Introduced by Threads

Single-threaded programs incur neither scheduling nor synchronization overhead, and need not use locks to preserve the consistency of data structures. Scheduling and interthread coordination have performance costs; for threads to offer a performance improvement, the performance

benefits of parallelization must outweigh the costs introduced by concurrency.

11.3.1. Context Switching

If the main thread is the only schedulable thread, it will almost never be scheduled out. On the other hand, if there are more runnable threads than CPUs, eventually the OS will preempt one thread so that another can use the CPU. This causes a *context switch*, which requires saving the execution context of the currently running thread and restoring the execution context of the newly scheduled thread.

Context switches are not free; thread scheduling requires manipulating shared data structures in the OS and JVM. The OS and JVMuse the same CPUs your program does; more CPU time spent in JVM and OS code means less is available for your program. But OS and JVM activity is not the only cost of context switches. When a new thread is switched in, the data it needs is unlikely to be in the local processor cache, so a context switch causes a flurry of cache misses, and thus threads run a little more slowly when they are first scheduled. This is one of the reasons that schedulers give each runnable thread a certain minimum time quantum even when many other threads are waiting: it amortizes the cost of the context switch and its consequences over more uninterrupted execution time, improving overall throughput (at some cost to responsiveness).

Listing 11.2. Synchronization that has No Effect. *Don't do this.*

```
synchronized (new Object()) {
    // do something
}
```

When a thread blocks because it is waiting for a contended lock, the JVM usually suspends the thread and allows it to be switched out. If threads block frequently, they will be unable to use their full scheduling quantum. A program that does more blocking (blocking I/O, waiting for contended locks, or waiting on condition variables) incurs more context switches than one that is CPU-bound, increasing scheduling overhead and reducing throughput. (Nonblocking algorithms can also help reduce context switches; see **Chapter 15**.)

The actual cost of context switching varies across platforms, but a good rule of thumb is that a context switch costs the equivalent of 5,000 to 10,000 clock cycles, or several microseconds on most current processors.

The `vmstat` command on Unix systems and the `perfmon` tool on Windows systems report the number of context switches and the percentage of time spent in the kernel. High kernel usage (over 10%) often indicates heavy scheduling activity, which may be caused by blocking due to I/O or lock contention.

11.3.2. Memory Synchronization

The performance cost of synchronization comes from several sources. The visibility guarantees provided by `synchronized` and `volatile` may entail using special instructions called *memory barriers* that can flush or invalidate caches, flush hardware write buffers, and stall execution pipelines. Memory barriers may also have indirect performance consequences because they inhibit other compiler optimizations; most operations cannot be reordered with memory barriers.

When assessing the performance impact of synchronization, it is important to distinguish between *contended* and *uncontended* synchronization. The `synchronized` mechanism is optimized for the uncontended case (`volatile` is always uncontended), and at this writing, the performance cost of a "fast-path" uncontended synchronization ranges from 20 to 250 clock cycles for most systems. While this is certainly not zero, the effect of needed, uncontended synchronization is rarely significant in overall application performance, and the alternative involves compromising safety and potentially signing yourself (or your successor) up for some very painful bug hunting later.

Modern JVMs can reduce the cost of incidental synchronization by optimizing away locking that can be proven never to contend. If a lock object is accessible only to the current thread, the JVM is permitted to optimize away a lock acquisition because there is no way another thread could synchronize on the same lock. For example, the lock acquisition in **Listing 11.2** can always be eliminated by the JVM.

More sophisticated JVMs can use *escape analysis* to identify when a local object reference is never published to the heap and is therefore thread-local. In `getStoogeNames` in **Listing 11.3**, the only reference to the `List` is the local variable `stooges`, and stack-confined variables are automatically thread-local. A naive execution of `getStoogeNames` would acquire and release the lock on the `Vector` four times, once for each call to `add` or `toString`. However, a smart runtime compiler can inline these calls and then see that `stooges` and its internal state never escape, and therefore that all four lock acquisitions can be eliminated.[4]

[4] This compiler optimization, called *lock elision*, is performed by the IBM JVM and is expected in HotSpot as of Java 7.

Listing 11.3. Candidate for Lock Elision.

```
public String getStoogeNames() {
    List<String> stooges = new Vector<String>();
    stooges.add("Moe");
    stooges.add("Larry");
    stooges.add("Curly");
    return stooges.toString();
}
```

Even without escape analysis, compilers can also perform *lock coarsening*, the merging of adjacent `synchronized` blocks using the same lock. For `getStooge-Names`, a JVM that performs lock coarsening might combine the three calls to `add` and the call to `toString` into a single lock acquisition and release, using heuristics on the relative cost of synchronization versus the instructions inside the `synchronized` block.[5] Not only does this reduce the synchronization overhead, but it also gives the optimizer a much larger block to work with, likely enabling other optimizations.

[5] A smart dynamic compiler can figure out that this method always returns the same string, and after the first execution recompile `getStoogeNames` to simply return the value returned by the first execution.

Don't worry excessively about the cost of uncontended synchronization. The basic mechanism is already quite fast, and JVMs can perform addi-

tional optimizations that further reduce or eliminate the cost. Instead, focus optimization efforts on areas where lock contention actually occurs.

Synchronization by one thread can also affect the performance of other threads. Synchronization creates traffic on the shared memory bus; this bus has a limited bandwidth and is shared across all processors. If threads must compete for synchronization bandwidth, all threads using synchronization will suffer.[6]

[6] This aspect is sometimes used to argue against the use of nonblocking algorithms without some sort of backoff, because under heavy contention, nonblocking algorithms generate more synchronization traffic than lock-based ones. See **Chapter 15**.

11.3.3. Blocking

Uncontended synchronization can be handled entirely within the JVM (**Bacon et al., 1998**); contended synchronization may require OS activity, which adds to the cost. When locking is contended, the losing thread(s) must block. The JVM can implement blocking either via *spin-waiting* (repeatedly trying to acquire the lock until it succeeds) or by *suspending* the blocked thread through the operating system. Which is more efficient depends on the relationship between context switch overhead and the time until the lock becomes available; spin-waiting is preferable for short waits and suspension is preferable for long waits. Some JVMs choose between the two adaptively based on profiling data of past wait times, but most just suspend threads waiting for a lock.

Suspending a thread because it could not get a lock, or because it blocked on a condition wait or blocking I/O operation, entails two additional context switches and all the attendant OS and cache activity: the blocked thread is switched out before its quantum has expired, and is then switched back in later after the lock or other resource becomes available. (Blocking due to lock contention also has a cost for the thread holding the lock: when it releases the lock, it must then ask the OS to resume the blocked thread.)

11.4. Reducing Lock Contention

We've seen that serialization hurts scalability and that context switches hurt performance. Contended locking causes both, so reducing lock contention can improve both performance and scalability.

Access to resources guarded by an exclusive lock is serialized—only one thread at a time may access it. Of course, we use locks for good reasons, such as preventing data corruption, but this safety comes at a price. Persistent contention for a lock limits scalability.

The principal threat to scalability in concurrent applications is the exclusive resource lock.

Two factors influence the likelihood of contention for a lock: how often that lock is requested and how long it is held once acquired.[7] If the product of these factors is sufficiently small, then most attempts to acquire the lock will be uncontended, and lock contention will not pose a significant scalability impediment. If, however, the lock is in sufficiently high demand, threads will block waiting for it; in the extreme case, processors will sit idle even though there is plenty of work to do.

[7] This is a corollary of *Little's law*, a result from queueing theory that says "the average number of customers in a stable system is equal to their average arrival rate multiplied by their average time in the system". (**Little, 1961**)

There are three ways to reduce lock contention:

- Reduce the duration for which locks are held;
- Reduce the frequency with which locks are requested; or
- Replace exclusive locks with coordination mechanisms that permit greater concurrency.

11.4.1. Narrowing Lock Scope ("Get in, Get Out")

An effective way to reduce the likelihood of contention is to hold locks as briefly as possible. This can be done by moving code that doesn't require the lock out of `synchronized` blocks, especially for expensive operations and potentially blocking operations such as I/O.

It is easy to see how holding a "hot" lock for too long can limit scalability; we saw an example of this in `SynchronizedFactorizer` in **Chapter 2**. If an operation holds a lock for 2 milliseconds and every operation requires that lock, throughput can be no greater than 500 operations per second, no matter how many processors are available. Reducing the time the lock is held to 1 millisecond improves the lock-induced throughput limit to 1000 operations per second.[8]

[8] Actually, this calculation *understates* the cost of holding locks for too long because it doesn't take into account the context switch overhead generated by increased lock contention.

`AttributeStore` in **Listing 11.4** shows an example of holding a lock longer than necessary. The `userLocationMatches` method looks up the user's location in a `Map` and uses regular expression matching to see if the resulting value matches the supplied pattern. The entire `userLocationMatches` method is `synchronized`, but the only portion of the code that actually needs the lock is the call to `Map.get`.

Listing 11.4. Holding a lock longer than necessary.

```
@ThreadSafe
public class AttributeStore {
    @GuardedBy("this") private final Map<String, String>
            attributes = new HashMap<String, String>();

    public synchronized boolean userLocationMatches(String name,
                                                    String regexp) {
        String key = "users." + name + ".location";
        String location = attributes.get(key);
        if (location == null)
            return false;
        else
            return Pattern.matches(regexp, location);
    }
}
```

`BetterAttributeStore` in **Listing 11.5** rewrites `AttributeStore` to reduce significantly the lock duration. The first step is to construct the `Map` key associated with the user's location, a string of the form `users.`*name*`.location`. This entails instantiating a `StringBuilder` object, appending several strings to it, and instantiating the result as a `String`. After the location has been retrieved, the regular expression is matched against the resulting location string. Because constructing the key string and processing the regular expression do not access shared state, they need not be executed with the lock held. `BetterAttributeStore` factors these steps out of the `synchronized` block, thus reducing the time the lock is held.

Listing 11.5. Reducing Lock Duration.

```
@ThreadSafe
public class BetterAttributeStore {
    @GuardedBy("this") private final Map<String, String>
            attributes = new HashMap<String, String>();

    public boolean userLocationMatches(String name, String regexp) {
        String key = "users." + name + ".location";
        String location;
        synchronized (this) {
            location = attributes.get(key);
        }
        if (location == null)
            return false;
        else
            return Pattern.matches(regexp, location);
    }
}
```

Reducing the scope of the lock in `userLocationMatches` substantially reduces the number of instructions that are executed with the lock held. By Amdahl's law, this removes an impediment to scalability because the amount of serialized code is reduced.

Because `AttributeStore` has only one state variable, `attributes`, we can improve it further by the technique of *delegating thread safety* (**Section 4.3**). By replacing `attributes` with a thread-safe `Map` (a `Hashtable`, `synchronizedMap`, or `ConcurrentHashMap`), `AttributeStore` can delegate all its thread safety obligations to the underlying thread-safe collection. This eliminates the need for explicit syn-

chronization in `AttributeStore`, reduces the lock scope to the duration of the `Map` access, and removes the risk that a future maintainer will undermine thread safety by forgetting to acquire the appropriate lock before accessing `attributes`.

While shrinking `synchronized` blocks can improve scalability, a `synchronized` block can be *too* small—operations that need to be atomic (such updating multiple variables that participate in an invariant) must be contained in a single `synchronized` block. And because the cost of synchronization is nonzero, breaking one `synchronized` block into multiple `synchronized` blocks (correctness permitting) at some point becomes counterproductive in terms of performance.[9] The ideal balance is of course platform-dependent, but in practice it makes sense to worry about the size of a `synchronized` block only when you can move "substantial" computation or blocking operations out of it.

[9] If the JVM performs lock coarsening, it may undo the splitting of `synchronized` blocks anyway.

11.4.2. Reducing Lock Granularity

The other way to reduce the fraction of time that a lock is held (and therefore the likelihood that it will be contended) is to have threads ask for it less often. This can be accomplished by *lock splitting* and *lock striping*, which involve using separate locks to guard multiple independent state variables previously guarded by a single lock. These techniques reduce the granularity at which locking occurs, potentially allowing greater scalability—but using more locks also increases the risk of deadlock.

As a thought experiment, imagine what would happen if there was *only one* lock for the entire application instead of a separate lock for each object. Then execution of all `synchronized` blocks, regardless of their lock, would be serialized. With many threads competing for the global lock, the chance that two threads want the lock at the same time increases, resulting in more contention. So if lock requests were instead distributed over a *larger* set of locks, there would be less contention. Fewer threads would be blocked waiting for locks, thus increasing scalability.

If a lock guards more than one *independent* state variable, you may be able to improve scalability by splitting it into multiple locks that each guard different variables. This results in each lock being requested less often.

`ServerStatus` in **Listing 11.6** shows a portion of the monitoring interface for a database server that maintains the set of currently logged-on users and the set of currently executing queries. As a user logs on or off or query execution begins or ends, the `ServerStatus` object is updated by calling the appropriate `add` or `remove` method. The two types of information are completely independent; `ServerStatus` could even be split into two separate classes with no loss of functionality.

Instead of guarding both `users` and `queries` with the `ServerStatus` lock, we can instead guard each with a separate lock, as shown in **Listing 11.7**. After splitting the lock, each new finer-grained lock will see less locking traffic than the original coarser lock would have. (Delegating to a thread-safe `Set` implementation for `users` and `queries` instead of using explicit synchronization would implicitly provide lock splitting, as each `Set` would use a different lock to guard its state.)

Splitting a lock into two offers the greatest possibility for improvement when the lock is experiencing moderate but not heavy contention. Splitting locks that are experiencing little contention yields little net improvement in performance or throughput, although it might increase the load threshold at which performance starts to degrade due to contention. Splitting locks experiencing moderate contention might actually turn them into mostly uncontended locks, which is the most desirable outcome for both performance and scalability.

Listing 11.6. Candidate for Lock Splitting.

```
@ThreadSafe
public class ServerStatus {
    @GuardedBy("this") public final Set<String> users;
    @GuardedBy("this") public final Set<String> queries;
    ...
    public synchronized void addUser(String u) { users.add(u); }
    public synchronized void addQuery(String q) { queries.add(q); }
    public synchronized void removeUser(String u) {
        users.remove(u);
    }
    public synchronized void removeQuery(String q) {
        queries.remove(q);
    }
}
```

Listing 11.7. `ServerStatus` **Refactored to Use Split Locks.**

```
@ThreadSafe
public class ServerStatus {
    @GuardedBy("users") public final Set<String> users;
    @GuardedBy("queries") public final Set<String> queries;
    ...
    public void addUser(String u) {
        synchronized (users) {
            users.add(u);
        }
    }

    public void addQuery(String q) {
        synchronized (queries) {
            queries.add(q);
        }
    }
    // remove methods similarly refactored to use split locks
}
```

11.4.3. Lock Striping

Splitting a heavily contended lock into two is likely to result in two heavily contended locks. While this will produce a small scalability improvement by enabling two threads to execute concurrently instead of one, it still does not dramatically improve prospects for concurrency on a system with many processors. The lock splitting example in the `ServerStatus` classes does not offer any obvious opportunity for splitting the locks further.

Lock splitting can sometimes be extended to partition locking on a variablesized set of independent objects, in which case it is called *lock striping*. For example, the implementation of `ConcurrentHashMap` uses an array of 16 locks, each of which guards 1/16 of the hash buckets; bucket *N* is guarded by lock *N* mod 16. Assuming the hash function provides reasonable spreading characteristics and keys are accessed uniformly, this should reduce the demand for any given lock by approximately a factor of 16. It is this technique that enables `ConcurrentHashMap` to support up to 16 concurrent writers. (The number of locks could be increased to provide even better concurrency under heavy access on high-processor-count systems, but the number of stripes should be increased beyond the default of 16 only when you have evidence that concurrent writers are generating enough contention to warrant raising the limit.)

One of the downsides of lock striping is that locking the collection for exclusive access is more difficult and costly than with a single lock. Usually an operation can be performed by acquiring at most one lock, but occasionally you need to lock the entire collection, as when `ConcurrentHashMap` needs to expand the map and rehash the values into a larger set of buckets. This is typically done by acquiring all of the locks in the stripe set.[10]

[10] The only way to acquire an arbitrary set of intrinsic locks is via recursion.

`StripedMap` in **Listing 11.8** illustrates implementing a hash-based map using lock striping. There are `N_LOCKS` locks, each guarding a subset of the buckets. Most methods, like `get`, need acquire only a single bucket lock. Some methods may need to acquire all the locks but, as in the implementation for `clear`, may not need to acquire them all simultaneously. [11]

[11] Clearing the `Map` in this way is not atomic, so there is not necessarily a time when the `Striped-Map` is actually empty if other threads are concurrently adding elements; making the operation atomic would require acquiring all the locks at once. However, for concurrent collections that clients typically cannot lock for exclusive access, the result of methods like `size` or `isEmpty` may be out of date by the time they return any-

way, so this behavior, while perhaps somewhat surprising, is usually acceptable.

11.4.4. Avoiding Hot Fields

Lock splitting and lock striping can improve scalability because they enable different threads to operate on different data (or different portions of the same data structure) without interfering with each other. A program that would benefit from lock splitting necessarily exhibits contention for a *lock* more often than for the *data* guarded by that lock. If a lock guards two independent variables *X* and *Y*, and thread *A* wants to access *X* while *B* wants to access *Y* (as would be the case if one thread called `addUser` while another called `addQuery` in `ServerStatus`), then the two threads are not contending for any data, even though they are contending for a lock.

Listing 11.8. Hash-based Map Using Lock Striping.

```
@ThreadSafe
public class StripedMap {
    // Synchronization policy: buckets[n] guarded by locks[n%N_LOCKS]
    private static final int N_LOCKS = 16;
    private final Node[] buckets;
    private final Object[] locks;

    private static class Node { ... }

    public StripedMap(int numBuckets) {
        buckets = new Node[numBuckets];
        locks = new Object[N_LOCKS];
        for (int i = 0; i < N_LOCKS; i++)
            locks[i] = new Object();
    }

    private final int hash(Object key) {
        return Math.abs(key.hashCode() % buckets.length);
    }

    public Object get(Object key) {
        int hash = hash(key);
        synchronized (locks[hash % N_LOCKS]) {
            for (Node m = buckets[hash]; m != null; m = m.next)
                if (m.key.equals(key))
                    return m.value;
        }
        return null;
    }

    public void clear() {
        for (int i = 0; i < buckets.length; i++) {
            synchronized (locks[i % N_LOCKS]) {
                buckets[i] = null;
            }
        }
    }
    ...
}
```

Lock granularity cannot be reduced when there are variables that are required for every operation. This is yet another area where raw performance and scalability are often at odds with each other; common optimizations such as caching frequently computed values can introduce “hot fields” that limit scalability.

If you were implementing `HashMap`, you would have a choice of how `size` computes the number of entries in the `Map`. The simplest approach is to count the number of entries every time it is called. A common optimization is to update a separate counter as entries are added or removed; this slightly increases the cost of a `put` or `remove` operation to keep the counter up-to-date, but reduces the cost of the `size` operation from $O(n)$ to $O(1)$.

Keeping a separate count to speed up operations like `size` and `isEmpty` works fine for a single-threaded or fully synchronized implementation, but makes it much harder to improve the scalability of the implementation because every operation that modifies the map must now update the shared counter. Even if you use lock striping for the hash chains, synchronizing access to the counter reintroduces the scalability problems of exclusive locking. What looked like a performance optimization—caching the results of the `size` operation—has turned into a scalability liability. In this case, the counter is called a *hot field* because every mutative operation needs to access it.

`ConcurrentHashMap` avoids this problem by having `size` enumerate the stripes and add up the number of elements in each stripe, instead of maintaining a global count. To avoid enumerating every element, `ConcurrentHashMap` maintains a separate count field for each stripe, also guarded by the stripe lock.[12]

[12] If `size` is called frequently compared to mutative operations, striped data structures can optimize for this by caching the collection size in a `volatile` whenever `size` is called and invalidating the cache (setting it to -1) whenever the collection is modified. If the cached value is nonnegative on entry to `size`, it is accurate and can be returned; otherwise it is recomputed.

11.4.5. Alternatives to Exclusive Locks

A third technique for mitigating the effect of lock contention is to forego the use of exclusive locks in favor of a more concurrency-friendly means of managing shared state. These include using the concurrent collections, read-write locks, immutable objects and atomic variables.

`ReadWriteLock` (see **Chapter 13**) enforces a multiple-reader, single-writer locking discipline: more than one reader can access the shared resource concurrently so long as none of them wants to modify it, but writers must acquire the lock excusively. For read-mostly data structures, `ReadWriteLock` can offer greater concurrency than exclusive locking; for read-only data structures, immutability can eliminate the need for locking entirely.

Atomic variables (see **Chapter 15**) offer a means of reducing the cost of updating "hot fields" such as statistics counters, sequence generators, or the reference to the first node in a linked data structure. (We used `AtomicLong` to maintain the hit counter in the servlet examples in **Chapter 2**.) The atomic variable classes provide very fine-grained (and thereforemore scalable) atomic operations on integers or object references, and are implemented using low-level concurrency primitives (such as compare-and-swap) provided by most modern processors. If your class has a small number of hot fields that do not participate in invariants with other variables, replacing them with atomic variables may improve scalability. (Changing your algorithm to have fewer hot fields might improve scalability even more—atomic variables reduce the cost of updating hot fields, but they don't eliminate it.)

11.4.6. Monitoring CPU Utilization

When testing for scalability, the goal is usually to keep the processors fully utilized. Tools like `vmstat` and `mpstat` on Unix systems or `perfmon` on Windows systems can tell you just how "hot" the processors are running.

If the CPUs are asymmetrically utilized (some CPUs are running hot but others are not) your first goal should be to find increased parallelism in your program. Asymmetric utilization indicates that most of the computation is going on in a small set of threads, and your application will not be able to take advantage of additional processors.

If the CPUs are not fully utilized, you need to figure out why. There are several likely causes:

Insufficent load. It may be that the application being tested is just not subjected to enough load. You can test for this by increasing the load and measuring changes in utilization, response time, or service time. Generating enough load to saturate an application can require substantial computer power; the problem may be that the client systems, not the system being tested, are running at capacity.

I/O-bound. You can determine whether an application is disk-bound using `iostat` or `perfmon`, and whether it is bandwidth-limited by monitoring traffic levels on your network.

Externally bound. If your application depends on external services such as a database or web service, the bottleneck may not be in your code. You can test for this by using a profiler or database administration tools to determine how much time is being spent waiting for answers from the external service.

Lock contention. Profiling tools can tell you how much lock contention your application is experiencing and which locks are "hot". You can often get the same information without a profiler through random sampling, triggering a few thread dumps and looking for threads contending for locks. If a thread is blocked waiting for a lock, the appropriate stack frame in the thread dump indicates "waiting to lock monitor . . . " Locks that are mostly uncontended rarely show up in a thread dump; a heavily contended lock will almost always have at least one thread waiting to acquire it and so will frequently appear in thread dumps.

If your application is keeping the CPUs sufficiently hot, you can use monitoring tools to infer whether it would benefit from additional CPUs. A program with only four threads may be able to keep a 4-way system fully utilized, but is unlikely to see a performance boost if moved to an 8-way system since there would need to be waiting runnable threads to take advantage of the additional processors. (You may also be able to reconfigure the program to divide its workload over more threads, such as adjusting a thread pool size.) One of the columns reported by `vmstat` is the number of threads that are runnable but not currently running because a CPU is not available; if CPU utilization is high and there are always runnable threads waiting for a CPU, your application would probably benefit from more processors.

11.4.7. Just Say No to Object Pooling

In early JVM versions, object allocation and garbage collection were slow, [13] but their performance has improved substantially since then. In fact, allocation in Java is now faster than `malloc` is in C: the common code

path for `new Object` in HotSpot 1.4.x and 5.0 is approximately ten machine instructions.

[13] As was everything else—synchronization, graphics, JVM startup, reflection—predictably so in the first version of an experimental technology.

To work around "slow" object lifecycles, many developers turned to object pooling, where objects are recycled instead of being garbage collected and allocated anew when needed. Even taking into account its reduced garbage collection overhead, object pooling has been shown to be a performance loss[14] for all but the most expensive objects (and a serious loss for light- and medium-weight objects) in single-threaded programs (**Click, 2005**).

[14] In addition to being a loss in terms of CPU cycles, object pooling has a number of other problems, among them the challenge of setting pool sizes correctly (too small, and pooling has no effect; too large, and it puts pressure on the garbage collector, retaining memory that could be used more effectively for something else); the risk that an object will not be properly reset to its newly allocated state, introducing subtle bugs; the risk that a thread will return an object to the pool but continue using it; and that it makes more work for generational garbage collectors by encouraging a pattern of old-to-young references.

In concurrent applications, pooling fares even worse. When threads allocate new objects, very little inter-thread coordination is required, as allocators typically use thread-local allocation blocks to eliminate most synchronization on heap data structures. But if those threads instead request an object from a pool, some synchronization is necessary to coordinate access to the pool data structure, creating the possibility that a thread will block. Because blocking a thread due to lock contention is hundreds of times more expensive than an allocation, even a small amount of pool-induced contention would be a scalability bottleneck. (Even an uncontended synchronization is usually more expensive than allocating an object.) This is yet another technique intended as a performance optimization but that turned into a scalability hazard. Pooling has its uses,[15] but is of limited utility as a performance optimization.

[15] In constrained environments, such as some J2ME or RTSJ targets, object pooling may still be required for effective memory management or to manage responsiveness.

Allocating objects is usually cheaper than synchronizing.

11.5. Example: Comparing Map Performance

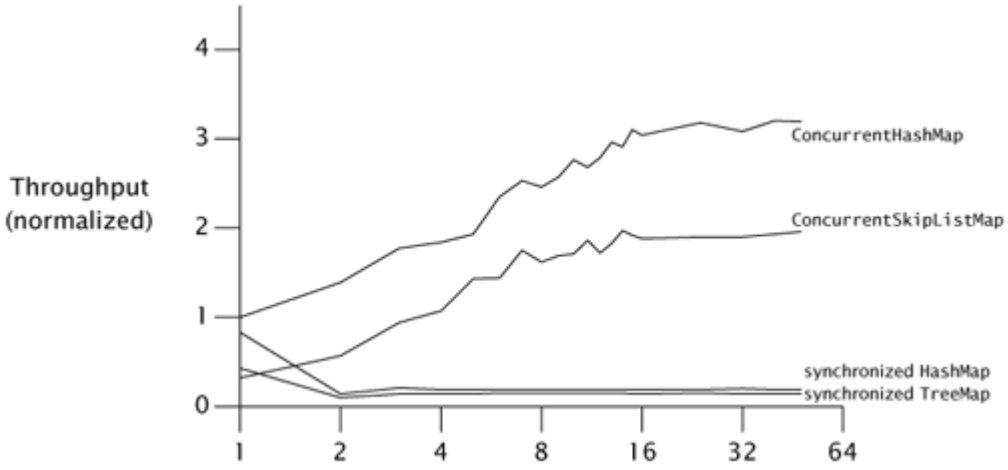
The single-threaded performance of `ConcurrentHashMap` is slightly better than that of a synchronized `HashMap`, but it is in concurrent use that it really shines. The implementation of `ConcurrentHashMap` assumes the most common operation is retrieving a value that already exists, and is therefore optimized to provide highest performance and concurrency for successful `get` operations.

The major scalability impediment for the synchronized `Map` implementations is that there is a single lock for the entire map, so only one thread can access the map at a time. On the other hand, `ConcurrentHashMap` does no locking for most successful read operations, and uses lock striping for write operations and those few read operations that do require locking. As a result, multiple threads can access the `Map` concurrently without blocking.

Figure 11.3 illustrates the differences in scalability between several `Map` implementations: `ConcurrentHashMap`, `ConcurrentSkipListMap`, and `HashMap` and `TreeMap` wrapped with `synchronizedMap`. The first two are thread-safe by design; the latter two are made thread-safe by the synchronized wrapper. In each run, *N* threads concurrently execute a tight loop that selects a random key and attempts to retrieve the value corresponding to that key. If the value is not present, it is added to the `Map` with probability *p* = .6, and if it is present, is removed with probability *p* = .02. The tests were run under a pre-release build of Java 6 on an 8-way Sparc V880, and the graph displays throughput normalized to the onethread case for `ConcurrentHashMap`. (The scalability gap between the concurrent and synchronized collections is even larger on Java 5.0.)

The data for `ConcurrentHashMap` and `ConcurrentSkipListMap` shows that they scale well to large numbers of threads; throughput continues to improve as threads are added. While the numbers of threads in **Figure 11.3** may not seem large, this test program generates more contention per thread than a typical application because it does little other than pound on the `Map`; a real program would do additional thread-local work in each iteration.

Figure 11.3. Comparing Scalability of `Map` Implementations.



The numbers for the synchronized collections are not as encouraging. Performance for the one-thread case is comparable to `ConcurrentHashMap`, but once the load transitions from mostly uncontended to mostly contended—which happens here at two threads—the synchronized collections suffer badly. This is common behavior for code whose scalability is limited by lock contention. So long as contention is low, time per operation is dominated by the time to actually do the work and throughput may improve as threads are added. Once contention becomes significant, time per operation is dominated by context switch and scheduling delays, and adding more threads has little effect on throughput.

11.6. Reducing Context Switch Overhead

Many tasks involve operations that may block; transitioning between the running and blocked states entails a context switch. One source of blocking in server applications is generating log messages in the course of processing requests; to illustrate how throughput can be improved by reduc-

ing context switches, we'll analyze the scheduling behavior of two logging approaches.

Most logging frameworks are thin wrappers around `println`; when you have something to log, just write it out right then and there. Another approach was shown in `LogWriter` on page 152: the logging is performed in a dedicated background thread instead of by the requesting thread. From the developer's perspective, both approaches are roughly equivalent. But there may be a difference in performance, depending on the volume of logging activity, how many threads are doing logging, and other factors such as the cost of context switching.[16]

[16] Building a logger that moves the I/O to another thread may improve performance, but it also introduces a number of design complications, such as interruption (what happens if a thread blocked in a logging operation is interrupted?), service guarantees (does the logger guarantee that a successfully queued log message will be logged prior to service shutdown?), saturation policy (what happens when the producers log messages faster than the logger thread can handle them?), and service lifecycle (how do we shut down the logger, and how do we communicate the service state to producers?).

The service time for a logging operation includes whatever computation is associated with the I/O stream classes; if the I/O operation blocks, it also includes the duration for which the thread is blocked. The operating system will deschedule the blocked thread until the I/O completes, and probably a little longer. When the I/O completes, other threads are probably active and will be allowed to finish out their scheduling quanta, and threads may already be waiting ahead of us on the scheduling queue—further adding to service time. Alternatively, if multiple threads are logging simultaneously, theremay be contention for the output stream lock, in which case the result is the same as with blocking I/O—the thread blocks waiting for the lock and gets switched out. Inline logging involves I/O and locking, which can lead to increased context switching and therefore increased service times.

Increasing request service time is undesirable for several reasons. First, service time affects quality of service: longer service times mean some-

one is waiting longer for a result. But more significantly, longer service times in this case mean more lock contention. The “get in, get out” principle of **Section 11.4.1** tells us that we should hold locks as briefly as possible, because the longer a lock is held, the more likely that lock will be contended. If a thread blocks waiting for I/O while holding a lock, another thread is more likely to want the lock while the first thread is holding it. Concurrent systems perform much better when most lock acquisitions are uncontended, because contended lock acquisition means more context switches. A coding style that encourages more context switches thus yields lower overall throughput.

Moving the I/O out of the request-processing thread is likely to shorten the mean service time for request processing. Threads calling `log` no longer block waiting for the output stream lock or for I/O to complete; they need only queue the message and can then return to their task. On the other hand, we've introduced the possibility of contention for the message queue, but the `put` operation is lighter-weight than the logging I/O (which might require system calls) and so is less likely to block in actual use (as long as the queue is not full). Because the request thread is now less likely to block, it is less likely to be context-switched out in the middle of a request. What we've done is turned a complicated and uncertain code path involving I/O and possible lock contention into a straight-line code path.

To some extent, we are just moving the work around, moving the I/O to a thread where its cost isn't perceived by the user (which may in itself be a win). But by moving *all* the logging I/O to a single thread, we also eliminate the chance of contention for the output stream and thus eliminate a source of blocking. This improves overall throughput because fewer resources are consumed in scheduling, context switching, and lock management.

Moving the I/O from many request-processing threads to a single logger thread is similar to the difference between a bucket brigade and a collection of individuals fighting a fire. In the “hundred guys running around with buckets” approach, you have a greater chance of contention at the water source and at the fire (resulting in overall less water delivered to

the fire), plus greater inefficiency because each worker is continuously switching modes (filling, running, dumping, running, etc.). In the bucket-brigade approach, the flow of water from the source to the burning building is constant, less energy is expended transporting the water to the fire, and each worker focuses on doing one job continuously. Just as interruptions are disruptive and productivity-reducing to humans, blocking and context switching are disruptive to threads.

Summary

Because one of the most common reasons to use threads is to exploit multiple processors, in discussing the performance of concurrent applications, we are usually more concerned with throughput or scalability than we are with raw service time. Amdahl's law tells us that the scalability of an application is driven by the proportion of code that must be executed serially. Since the primary source of serialization in Java programs is the exclusive resource lock, scalability can often be improved by spending less time holding locks, either by reducing lock granularity, reducing the duration for which locks are held, or replacing exclusive locks with nonexclusive or nonblocking alternatives.